

Learning to Decode: Graph Attention Network-Gated Recurrent Unit as Channel Decoder

Lixin Qian, Yang Lu[✉], *Senior Member, IEEE*, Xuefeng Wang[✉], Wei Chen[✉], *Senior Member, IEEE*,
and Bo Ai[✉], *Fellow, IEEE*

Abstract—Training deep learning (DL) models to address channel coding tasks has drawn increasing attention, a paradigm referred to as “learning to decode”. By constructing Tanner graphs based on parity-check matrices, graph neural networks (GNNs) have shown great promise, yet their performance can be further enhanced with more sophisticated neural architectures. This letter proposes a novel GNN-based decoder that enhances the traditional message-passing framework through the integration of a graph attention network (GAT) and a gated recurrent unit (GRU), termed GAT-GRU. Extensive experiments validate that the proposed GAT-GRU decoder outperforms conventional belief propagation (BP), weighted BP (WBP), and GNN decoders across Bose-Chaudhuri-Hocquenghem (BCH), regular low-density parity-check (LDPC), and 5G NR LDPC codes. Furthermore, GAT-GRU exhibits robust scalability to unseen code lengths and code rates, particularly for LDPC codes.

Index Terms—Learning to decode, GNNs, channel decoding, BP, BCH, LDPC.

I. INTRODUCTION

CHANNEL coding is a core technology of modern wireless communication systems, designed to reliably recover transmitted information with minimal error rates and latency over noisy channels. Prominent error-correcting codes, such as low-density parity-check (LDPC) [1] and Bose-Chaudhuri-Hocquenghem (BCH) codes, have been integrated into numerous standards, including 5G New Radio (NR) standards, 3GPP TS 38.212. These codes are commonly represented by Tanner graphs [2], upon which the most representative decoding algorithm is the belief propagation (BP) algorithm. As demonstrated in [3], BP is able to achieve maximum a posteriori (MAP) optimality on cycle-free graphs. However, Tanner graphs of practical codes inevitably contain cycles, particularly for short-to-moderate block lengths. These cycles violate the statistical independence assumption underlying BP, leading to message correlation and oscillation. This results in suboptimal convergence and prevents the decoder from approaching the maximum likelihood (ML) performance bound [4], thereby limiting the error correction capability.

Received 22 March 2026; accepted 26 April 2026. Date of publication 29 April 2026; date of current version 8 May 2026. This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grant 62571023, Grant U2468201, and Grant 62221001; and in part by Beijing Natural Science Foundation under Grant L242086. The associate editor coordinating the review of this article and approving it for publication was M. Hua. (*Corresponding author: Yang Lu.*)

Lixin Qian and Yang Lu are with the State Key Laboratory of Advanced Rail Autonomous Operation and the School of Computer Science and Technology, Beijing Jiaotong University, Beijing 100044, China (e-mail: 22331101@bjtu.edu.cn; yanglu@bjtu.edu.cn).

Xuefeng Wang, Wei Chen, and Bo Ai are with the School of Electronic and Information Engineering, Beijing Jiaotong University, Beijing 100044, China (e-mail: 23125060@bjtu.edu.cn; weich@bjtu.edu.cn; boai@bjtu.edu.cn).

Digital Object Identifier 10.1109/LWC.2026.3688747

Extensive research efforts have focused on improving the BP algorithm. The min-sum (MS) algorithm reduced decoding complexity at the cost of a minor performance trade-off, while the offset min-sum (OMS) and normalized min-sum (NMS) algorithms achieved favorable decoding performance with low complexity [5]. Further, deep learning (DL) has been exploited to boost wireless intelligence [6]. Regarding channel decoding, DL gives rise to an innovative paradigm, termed “learning to decode”. Specifically, there are two mainstream DL frameworks, namely end-to-end frameworks and BP-iteration frameworks. For instance, the authors in [7] and [8] employed the multi-layer perceptron (MLP) and the recurrent neural network (RNN) to develop end-to-end decoders, respectively, which achieved MAP-level bit error ratio (BER) performance for short-length codes, but they were limited by the curse of dimensionality and the inability to adapt to variant code lengths. Additionally, the authors in [9] proposed a Transformer-based end-to-end decoder, which however, slightly outperformed the single-iteration BP decoder. On the other hand, the authors in [10] adopted the BP-iteration framework by assigning learnable weights to the edges of the Tanner graph, demonstrating improved performance over BP for various high-density parity-check codes. To exploit the graph structure inherent in codes, the authors in [11] proposed to replace BP iterations with graph neural networks (GNNs) [12] on linear binary block codes, which achieved competitive performance with BP and exhibited scalability to varying code parameters.

Nevertheless, existing DL-based decoders demonstrate limited generalization capability. This letter proposes an augmented GNN-based decoder that integrates two powerful components: a graph attention network (GAT) [13] and a gated recurrent unit (GRU) [14], hence termed GAT-GRU. Specifically, we focus on a classic channel coding task involving a single-antenna transceiver over a noisy channel, with the BER adopted as the performance metric. Following the BP-iteration framework, we redesign the initialization step and propose to replace BP iterations with GAT-GRU iterations. The GAT-GRU decoder is trained using a supervised loss function that accounts for the outputs of all iterations. We conduct extensive experiments over BCH, regular LDPC, and 5G NR LDPC codes, yielding three key observations: 1) GAT-GRU generally outperforms BP [3], WBP [10], and GNN [11]; 2) GAT-GRU exhibits robust scalability to unseen code lengths and code rates for LDPC codes; 3) the inference speed of GNN-based decoders remains a bottleneck requiring further improvement. Additionally, ablation studies confirm the effectiveness of both GAT and GRU in enhancing decoding performance.

II. PROBLEM DEFINITION AND CONVENTIONAL METHOD

A. Problem Definition

Denote the original information consisting of k bits as $\mathbf{b} \in \mathbb{F}_2^k$, where $\mathbb{F}_2$ represents the binary Galois field. The transmitter encodes $\mathbf{b}$ into a codeword $\mathbf{c} \in \mathbb{F}_2^n$ via $\mathbf{c} = \mathbf{b}^T \mathbf{G}$, where $\mathbf{G} \in \mathbb{F}_2^{k \times n}$ denotes the generator matrix. A codeword is valid *iff* it holds that $\mathbf{H}\mathbf{c} = \mathbf{0}_{n-k}$, where $\mathbf{H} \in \mathbb{F}_2^{(n-k) \times n}$ denotes the binary parity-check matrix [11]. Notably, for coding schemes such as LDPC and BCH codes, both $\mathbf{G}$ and $\mathbf{H}$ are of full rank.

For Binary Phase Shift Keying (BPSK), $\mathbf{c}$ is modulated to form the transmit signal $\mathbf{x} \in \mathbb{R}^n$ as follows:

$$\mathbf{x} = -2\mathbf{c} + \mathbf{1}_n \in \{-1, 1\}^n. \quad (1)$$

Subsequently, the received signal $\mathbf{y} \in \mathbb{R}^n$ is given by

$$\mathbf{y} = \mathbf{x} + \mathbf{n}, \quad (2)$$

where $\mathbf{n} \sim \mathcal{N}(\mathbf{0}, \epsilon_{\text{ch}}^2 \mathbf{I}_n)$ represents additive white Gaussian noise (AWGN) with ϵ_{ch}^2 denoting the noise power.

A channel decoding task entails enabling the receiver to obtain an estimation of $\mathbf{c}$, denoted by $\hat{\mathbf{c}} \in \mathbb{F}_2^n$, based on $\mathbf{y}$ and pre-defined $\mathbf{H}$. Mathematically, we formulate the channel decoding process as a mapping given by:

$$\hat{\mathbf{c}} = \Phi_{\boldsymbol{\theta}}(\mathbf{y}, \mathbf{H}), \quad (3)$$

where $\boldsymbol{\theta}$ denotes the decision variables. The corresponding task is to construct $\Phi_{\boldsymbol{\theta}}(\cdot)$ and find the optimal $\boldsymbol{\theta}^*$ such that

$$\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} \text{BER}(\boldsymbol{\theta}) \triangleq \frac{1}{n} \|\mathbf{c} - \Phi_{\boldsymbol{\theta}}(\mathbf{y}, \mathbf{H})\|, \quad (4)$$

where $\text{BER}(\cdot)$ represents the BER.

B. Conventional Method

Traditionally, the BP algorithm has been widely employed for channel decoding. The overall process consists of four main stages: Tanner graph construction, initialization, BP iterations, and hard decision.

1) *Tanner Graph Construction*: A Tanner graph is constructed based on $\mathbf{H}$ and defined as a bipartite graph $\mathcal{G} = (\mathcal{V} \cup \mathcal{U}, \mathcal{E})$, where $\mathcal{V} \triangleq \{v_1, v_2, \dots, v_n\}$ denotes the set of variable nodes (VNs) and $\mathcal{U} \triangleq \{u_1, u_2, \dots, u_{n-k}\}$ denotes the set of check nodes (CNs) [3]. Each v_i corresponds to a codeword bit $\mathbf{c}_{[i]}$, while each u_j corresponds to a parity check equation defined by $\mathbf{H}_{[j,:]} \mathbf{c} = 0$. An edge $\langle v_i, u_j \rangle \in \mathcal{E}$ *iff* $\mathbf{H}_{[j,i]} = 1$. We denote the set of neighboring CNs for v_i as $\mathcal{N}(v_i)$, and the set of neighboring VNs for u_j as $\mathcal{N}(u_j)$.

2) *Initialization*: Denote the initial feature of v_i as $L_{v_i}^{(0)}$, which is computed as

$$L_{v_i}^{(0)} = \text{LLR}(\mathbf{y}_{[i]}) \triangleq \ln \left(\frac{\text{Prob}(\mathbf{c}_{[i]} = 0 \mid \mathbf{y}_{[i]})}{\text{Prob}(\mathbf{c}_{[i]} = 1 \mid \mathbf{y}_{[i]})} \right) = \frac{2\mathbf{y}_{[i]}}{\epsilon_{\text{ch}}^2}, \quad (5)$$

where $\text{LLR}(\cdot)$ represents the log-likelihood ratio (LLR).

The messages from u_j to v_i and from v_i to u_j are denoted by $m_{u_j \rightarrow v_i}^{(0)}$ and $m_{v_i \rightarrow u_j}^{(0)}$, respectively, which are both initialized to 0.

3) *BP Iterations*: During the t -th BP iteration, the message $m_{u_j \rightarrow v_i}^{(t)}$ is computed as

$$m_{u_j \rightarrow v_i}^{(t)} = 2 \tanh^{-1} \left(\prod_{v'_i \in \mathcal{N}(u_j) \setminus \{v_i\}} \tanh \left(\frac{m_{v'_i \rightarrow u_j}^{(t-1)}}{2} \right) \right). \quad (6)$$

The message $m_{v_i \rightarrow u_j}^{(t)}$ is computed as

$$m_{v_i \rightarrow u_j}^{(t)} = L_{v_i}^{(0)} + \sum_{u'_j \in \mathcal{N}(v_i) \setminus \{u_j\}} m_{u'_j \rightarrow v_i}^{(t)}. \quad (7)$$

After T iterations, the output feature of v_i , which represents its final LLR estimation, is given by

$$L_{v_i}^{(T)} = L_{v_i}^{(0)} + \sum_{u_j \in \mathcal{N}(v_i)} m_{u_j \rightarrow v_i}^{(T)}. \quad (8)$$

4) *Hard Decision*: Based on the obtained $\{L_{v_i}^{(T)}\}$, a hard decision is performed to derive $\hat{\mathbf{c}}$ as

$$\hat{\mathbf{c}}_{[i]} = \begin{cases} 0, & \text{if } L_{v_i}^{(T)} \geq 0 \\ 1, & \text{if } L_{v_i}^{(T)} < 0 \end{cases}. \quad (9)$$

III. THE PROPOSED GAT-GRU DECODING APPROACH

This letter extends the BP-iteration framework with two key modifications: 1) redesigning the initialization procedure; and 2) replacing BP iterations with GAT-GRU iterations.

A. Initialization for GAT-GRU

The embeddings of v_i and u_j are initialized as $\mathbf{h}_{v_i}^{(0)} \in \mathbb{R}^{d_h}$ and $\mathbf{h}_{u_j}^{(0)} \in \mathbb{R}^{d_h}$, respectively, which are formulated as

$$\mathbf{h}_{v_i}^{(0)} = \phi_{\text{in}}(L_{v_i}^{(0)}), \quad \forall v_i \in \mathcal{V}, \quad (10)$$

$$\mathbf{h}_{u_j}^{(0)} = \mathbf{0}_{d_h}, \quad \forall u_j \in \mathcal{U}, \quad (11)$$

where $\phi_{\text{in}}(\cdot) : \mathbb{R} \rightarrow \mathbb{R}^{d_h}$ is implemented by a single-layer neural network, and d_h denotes a predefined dimension of the hidden embeddings.

B. GAT-GRU Iterations

Denote T' as the number of GAT-GRU iterations during the training phase. The t' -th ($t' \in \{1, 2, \dots, T'\}$) GAT-GRU iteration enables a mapping from $\{\mathbf{h}_{v_i}^{(t'-1)}, \mathbf{h}_{u_j}^{(t'-1)}\}$ to $\{L_{v_i}^{(t')}\}$, which consists of three components: GAT, GRU, and a readout process.

1) *GAT*: GAT consists of three sequential steps: message computation, attention score computation, and aggregation. For clarity, we take CN u_j as the target node through this subsection. Then, the embedding update of VN v_i follows the identical procedure but takes input of $\{\mathbf{h}_{u_j}^{(t')}, \mathbf{h}_{v_i}^{(t'-1)}\}_{u_j}$ and employs different learnable parameters.

A message from a neighboring node v_i to a target node u_j is computed by an MLP $\phi_{\text{msg}}(\cdot)$, which takes the concatenated embeddings of the two nodes from the previous iteration as input. The message from v_i to u_j is computed as

$$\mathbf{m}_{v_i \rightarrow u_j}^{(t')} = \phi_{\text{msg}} \left(\left[\mathbf{h}_{v_i}^{(t'-1)}, \mathbf{h}_{u_j}^{(t'-1)} \right] \right) \in \mathbb{R}^{d_m}, \quad (12)$$

where d_m denotes the message dimension, and $[\cdot, \cdot]$ represents the concatenation operation.

The attention score is computed as

$$\alpha_{v_i \rightarrow u_j}^{(t')} = \frac{\exp\left(\tau s_{v_i \rightarrow u_j}^{(t')}\right)}{\sum_{v'_i \in \mathcal{N}(u_j)} \exp\left(\tau s_{v'_i \rightarrow u_j}^{(t')}\right)}, \quad (13)$$

where τ is a learnable temperature parameter, and $\{s_{v_i \rightarrow u_j}^{(t')}\}_{v_i}$ denote the corresponding attention logits with $s_{v_i \rightarrow u_j}^{(t')}$ given by

$$s_{v_i \rightarrow u_j}^{(t')} = \phi_{\text{att}}\left(\left[\mathbf{h}_{u_j}^{(t'-1)}, \mathbf{h}_{v_i}^{(t'-1)}\right]\right) \in \mathbb{R}, \quad (14)$$

where $\phi_{\text{att}}(\cdot)$ is a dedicated MLP.

The attention-based message aggregated at u_j is given by

$$\tilde{\mathbf{m}}_{u_j}^{(t')} = \rho_{v_i \in \mathcal{N}(u_j)} \left(\theta \mathbf{m}_{v_i \rightarrow u_j}^{(t')} \odot \left(1 + \text{Clip}\left(g \alpha_{v_i \rightarrow u_j}^{(t')}, -\delta, \delta\right)\right) \right), \quad (15)$$

where $\odot$ denotes the Hadamard product, θ and g are trainable parameters, and $\text{Clip}(\cdot, -\delta, \delta)$ represents the clip operation, which restricts the modulated value to the interval $[-\delta, \delta]$. $\rho \in \{\text{Sum}, \text{Mean}, \text{Max}, \text{Min}\}$, which is a permutation-invariant operator and selected based on coding schemes. Then, $\tilde{\mathbf{m}}_{u_j}^{(t')}$ is processed by an adaptive normalization operator f_{norm} :

$$\tilde{\mathbf{m}}_{u_j}^{(t')} := f_{\text{norm}}\left(\tilde{\mathbf{m}}_{u_j}^{(t')}\right), \quad (16)$$

where f_{norm} can be layer normalization or a trainable scalar scaling, which is selected based on the density of $\mathbf{H}$.

2) *GRU*: The obtained node messages $\{\tilde{\mathbf{m}}_{u_j}^{(t')}\}$ are utilized to update node embeddings via GRU, which consists of two steps: candidate embedding computation and node embedding update.

The candidate embedding $\tilde{\mathbf{h}}_{u_j}^{(t')}$ is computed as

$$\tilde{\mathbf{h}}_{u_j}^{(t')} = \phi_{\text{h}}\left(\left[\tilde{\mathbf{m}}_{u_j}^{(t')}, \mathbf{r}_{u_j}^{(t')} \odot \mathbf{h}_{u_j}^{(t'-1)}\right]\right), \quad (17)$$

where $\phi_{\text{h}}(\cdot)$ is implemented by a two-layer MLP, and $\mathbf{r}_{u_j}^{(t')}$ denotes the reset gate, which is given by

$$\mathbf{r}_{u_j}^{(t')} = \phi_{\text{r}}\left(\left[\tilde{\mathbf{m}}_{u_j}^{(t')}, \mathbf{h}_{u_j}^{(t'-1)}\right]\right), \quad (18)$$

where $\phi_{\text{r}}(\cdot)$ is a two-layer MLP that maps $\mathbb{R}^{d_{\text{m}}+d_{\text{h}}} \rightarrow \mathbb{R}$.

Finally, the embedding of u_j is updated by

$$\mathbf{h}_{u_j}^{(t')} = (1 - \mathbf{z}_{u_j}^{(t')}) \odot \mathbf{h}_{u_j}^{(t'-1)} + \mathbf{z}_{u_j}^{(t')} \odot \tilde{\mathbf{h}}_{u_j}^{(t')}, \quad (19)$$

where $\mathbf{z}_{u_j}^{(t')}$ denotes the update gate, which is given by

$$\mathbf{z}_{u_j}^{(t')} = \phi_{\text{z}}\left(\left[\tilde{\mathbf{m}}_{u_j}^{(t')}, \mathbf{h}_{u_j}^{(t'-1)}\right]\right), \quad (20)$$

where $\phi_{\text{z}}(\cdot)$ is a two-layer MLP that maps $\mathbb{R}^{d_{\text{m}}+d_{\text{h}}} \rightarrow \mathbb{R}$.

3) *Readout*: The readout process recovers the VN embeddings back to LLRs, which is given by

$$L_{v_i}^{(t')} = \phi_{\text{out}}\left(\mathbf{h}_{v_i}^{(t')}\right), \quad (21)$$

where $\phi_{\text{out}}(\cdot) : \mathbb{R}^{d_{\text{h}}} \rightarrow \mathbb{R}$ is implemented by a single-layer neural network.

We cascade T' GAT-GRU iterations, with all GNNs sharing identical architecture and learnable parameters. Accordingly, we derive T' sets of LLR estimates, with the t' -th set being $\{L_{v_i}^{(t')}\}_{v_i}$. Particularly, all T' sets are utilized for model training, while only the T' -th set is fed into the hard decision (9) for codeword estimation.

C. Loss Function

The training loss function employs a multi-iteration weighted binary cross-entropy that assigns weights to the outputs of GAT-GRU iterations. The mathematical formulation of the loss function is given by

$$\mathcal{L} = - \sum_{t'=1}^{T'} w^{(t')} \frac{1}{n} \sum_{i=0}^{n-1} \left[\mathbf{c}_{[i]} \log(\sigma(L_{v_i}^{(t')})) + (1 - \mathbf{c}_{[i]}) \log(1 - \sigma(L_{v_i}^{(t')})) \right], \quad (22)$$

where $w^{(t')} = \frac{2t'}{T'(T'+1)}$ is the linearly increasing weight for the t' -th iteration, and $\sigma(\cdot)$ represents the Sigmoid function.

Remark 1: (Flexible implementation:) Although the decoder is trained with T' GAT-GRU iterations, we can flexibly implement an arbitrary number of GAT-GRU iterations in inference for better performance ($\geq T'$) or faster computation ($\leq T'$).

IV. NUMERICAL RESULTS

This section evaluates the proposed GAT-GRU decoder in terms of BER performance.

A. Experimental Setting

1) *Datasets*: We consider three types of coding schemes, i.e., BCH, regular LDPC, and 5G NR LDPC. For BCH codes, $\mathbf{H}$ is loaded directly from the NVIDIA Sionna [15]. For regular LDPC codes, $\mathbf{H}$ is randomly generated based on a specified VN degree of $v = 3$, CN degree of $c = 6$, and code length $n = 100$ [11]. For 5G NR LDPC codes, we utilize the NVIDIA Sionna to generate $\mathbf{H}$ by lifting and pruning one base graph defined in the 3GPP 5G NR standard, according to the code length and rate. Then, the training and test datasets are generated via the NVIDIA Sionna.

2) *Baselines*: Three BP-based decoders from existing works are adopted as baselines: BP [3], WBP [10], and GNN [11].

To quantitatively evaluate decoding performance, we measure the required energy per bit to noise power spectral density ratio (E_b/N_0) to achieve a target BER of 10^{-4} .

3) *Implementation Details*: All models are implemented in Python 3.8 with TensorFlow 2.8 and trained on a computer equipped with a single NVIDIA Tesla T4 GPU, managed by the NVIDIA driver version 470.223.02 with CUDA 11.4. The embedding dimensions for nodes and edges are set to $d_{\text{h}} = d_{\text{m}} = 20$ for BCH codes and $d_{\text{h}} = d_{\text{m}} = 16$ for LDPC codes, respectively. The weights of the node and message update functions are shared across all nodes and all iterations. For each training batch, the SNR is sampled uniformly in accordance with the parameters in Table I.

For BCH codes, we select layer normalization in (16). For LDPC codes, we use the trainable scalar scaling in (16). For 5G NR LDPC codes, rate-matching is inactivated during the training phase, while activated during the test phase.

B. Experimental Results

1) *Performance Over BCH Code*: Fig. 1 evaluates the BER performance of the GRU-GAT decoder and baselines over BCH codes with $(n, k) = (63, 45)$. For GNN and GAT-GRU, the results of $\{2, 4, 8\}$ iterations are reported with $T' = 8$,

TABLE I
TRAINING HYPERPARAMETERS

Parameter	BCH (63, 45)	5G NR LDPC (140, 60)	Regular LDPC
Training E_b/N_0 (dB)	[3, 8]	[2, 8]	[2, 4]
Batch Size	256	128	150
Learning rates	$1e-3 \rightarrow 1e-5$	$5e-4 \rightarrow 1e-5$	$2e-3 \rightarrow 1e-5$
Training iterations	{35k, 300k, 300k}	{35k, 300k, 300k}	{50k, 200k, 300k}

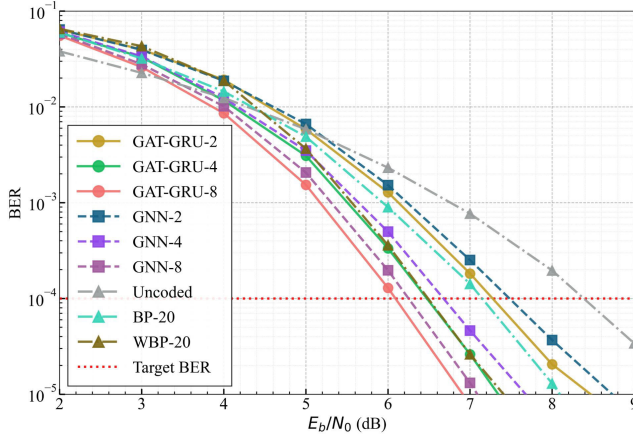


Fig. 1. BER performance over BCH codes with $(n, k) = (63, 45)$.

while for BP and WBP, the results are based on 20 iterations. As seen, 8-iteration GAT-GRU achieves the minimum BER across nearly the entire E_b/N_0 range. To achieve the target BER of 10^{-4} , 8-iteration GAT-GRU requires the minimum E_b/N_0 , achieving an E_b/N_0 reduction of 0.2, 0.5 and 1.3 dB compared to 8-iteration GNN, WBP and BP, respectively. Furthermore, it is remarkable that 4-iteration GAT-GRU still outperforms both BP and WBP. Notably, increasing the number of iterations leads to performance improvements for both GNN and GAT-GRU, but the performance gain tends to saturate.

2) *Performance Over 5G NR LDPC Code:* Figs. 2 and 3 illustrate the BER performance over 5G NR LDPC code, considering varying code lengths (n) and code rates ($R = k/n$), respectively.

Fig. 2 compares the BER performance of 10-iteration GAT-GRU, 10-iteration GNN, and 10-iteration BP. It is worth noting that both GNN and GAT-GRU are trained on codes with $(n, k) = (140, 60)$ but test on codes with $(n, k) \in \{(140, 60), (280, 120), (420, 180)\}$. To achieve the target BER of 10^{-4} , GAT-GRU achieves an E_b/N_0 reduction of 0.23 and 0.11 dB over BP and GNN, respectively, for $(n, k) = (140, 60)$. Regarding scalability to varying code lengths, GAT-GRU outperforms BP and GNN overall across the E_b/N_0 range, though it exhibits slightly inferior performance to BP at low E_b/N_0 levels. Nevertheless, when scaled to codes with $(n, k) = (420, 180)$, GAT-GRU still achieves an E_b/N_0 reduction of 0.14 dB over BP for the target BER.

Fig. 3 evaluates the scalability of the GAT-GRU decoder, which is trained with $(n, k, R) = (140, 60, 0.43)$ across

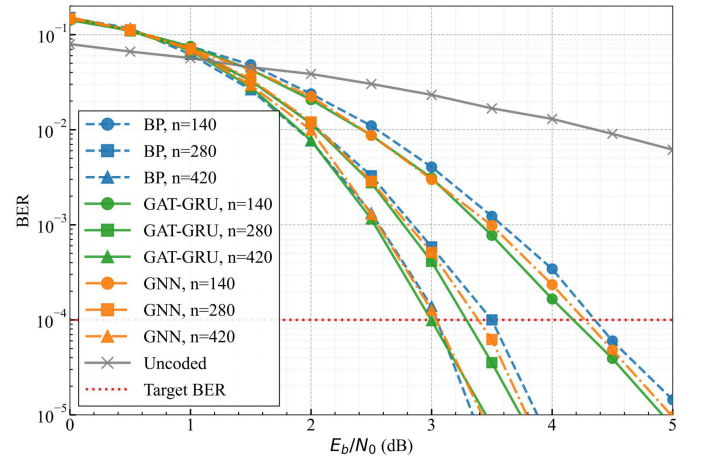


Fig. 2. BER performance over 5G NR LDPC codes with varying code lengths.

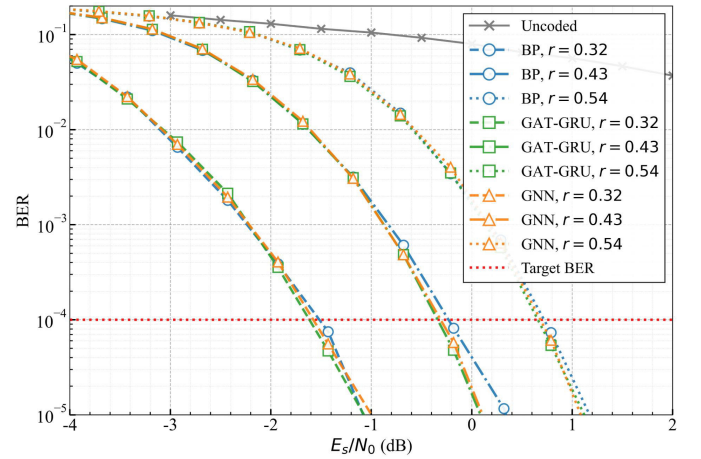


Fig. 3. BER performance over 5G NR LDPC codes with varying code rates.

different code lengths and code rates. For $(n, k, R) \in \{(280, 90, 0.32), (280, 120, 0.43), (280, 150, 0.54)\}$, the proposed GAT-GRU decoder consistently yields an E_b/N_0 reduction relative to GNN and BP to attain the target BER.

3) *Performance Over Regular LDPC Code:* Fig. 4 evaluates the BER performance across a range of code lengths $n \in \{100, 500, 1000\}$, where $k = n - \lfloor nv/c \rfloor$. All decoders are configured with 10 iterations, and GAT-GRU and GNN are trained with $n = 100$. It is observed that for every code length, GAT-GRU achieves superior decoding performance than GNN and BP, with particularly notable gains in the high E_b/N_0 region and for long code lengths. This result validates the scalability of GAT-GRU with respect to code length variations.

4) *Ablation Study:* Table II presents ablation studies to quantify the contributions of the GAT and GRU modules within GAT-GRU, where the evaluation metric is the required E_b/N_0 to achieve the target BER of 10^{-4} . For the ablation experiments: to isolate the contribution of GAT, we remove the GAT module. To isolate the contribution of GRU, we replace the GRU module with a standard MLP. In most cases, both the GAT and GRU modules contribute to E_b/N_0 reductions, even when scaling to varying code lengths. For different coding

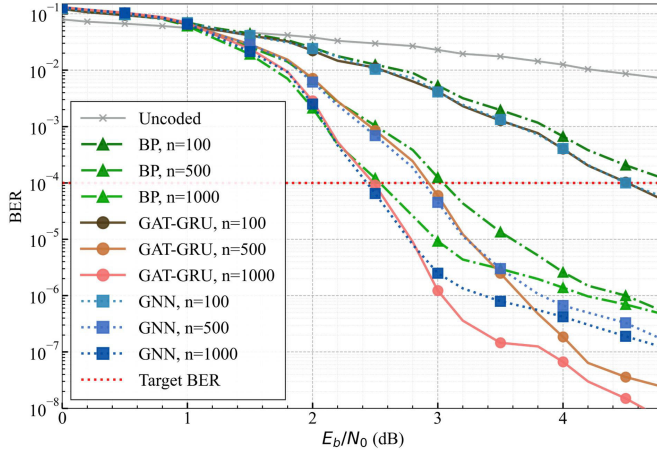


Fig. 4. BER performance over regular LDPC codes with varying code lengths.

TABLE II
ABLATION STUDIES

LDPC (n, k)	Baselines		Ablation		Proposed GAT-GRU
	BP	GNN	GRU	GAT-MLP	
(140, 60) [†]	4.48	4.36	4.32 (+.07)	4.29 (+.04)	4.25
(280, 120)	3.48	3.45	3.44 (+.03)	3.45 (+.04)	3.41
(420, 180)	3.14	3.11	3.06 (+.06)	3.08 (+.08)	3.00
(280, 90)	3.45	3.46	3.43 (+.04)	3.43 (+.04)	3.39
(280, 150)	3.49	3.46	3.45 (-.01)	3.46 (+.00)	3.46
BCH (63, 45)	7.39	6.30	6.16 (+.06)	6.25 (+.15)	6.10

[†]: (n, k) marked with [†] represents the training setting.

TABLE III
COMPLEXITY ANALYSIS OVER 5G NR LDPC CODES
WITH (n, k) = (140, 60)

Category	Decoder	Parameter count (K)	Throughput [§] (Mbps)	FLOPs (MFLOPs)
Baselines	BP	—	1.96	—
	GNN	18.9	0.26	119.4
Ablation	GRU	23.7	0.32	131.9
	GAT-MLP	17.5	0.22	140.6
Proposed	GAT-GRU	27.0	0.19	165.0

[§]: Throughput is measured on an NVIDIA Tesla T4 GPU.

schemes, the magnitude of the contributions from these two modules may vary. These results demonstrate that the proposed GAT-GRU decoder effectively leverages the strengths of GAT in spatial information aggregation and GRU in sequential iterative updating.

5) *Complexity Analysis*: Table III presents a complexity analysis of the proposed and baseline decoders. Compared to GNN, GAT-GRU exhibits a 42.5% increase in parameter count and a 38.7% increase in computational complexity. This complexity increment stems from a linear combination of the additional module costs, i.e., GAT and GRU. Both GNN and GAT-GRU achieve lower throughput than BP, which is consistent with the observations in [11]. Although the

effectiveness of GNN-based decoders across varying code lengths has been validated, additional efforts are required to accelerate inference from both algorithmic and hardware perspectives.

V. CONCLUSION

In this letter, we have proposed a GAT-GRU decoder that enhances the BP-iteration framework for binary linear codes. Two modules, i.e., GAT and GRU, were leveraged to enhance message passing over Tanner graphs constructed based on binary parity-check matrices. Numerical results have demonstrated that GAT-GRU achieves significant BER performance improvements over conventional BP-based decoders. A key finding is that GAT-GRU exhibits strong scalability for LDPC codes with respect to varying code lengths and rates. Future work will focus on striking a better balance between decoding performance and computational complexity, with the aim of accelerating the practical deployment of DL-based channel decoders.

ACKNOWLEDGMENT

The authors acknowledge the suggestions provided by Prof. Dusit Niyato during the preparation and revision of this letter.

REFERENCES

- [1] R. Gallager, "Low-density parity-check codes," *IRE Trans. Inf. Theory*, vol. 8, no. 1, pp. 21–28, 1962.
- [2] R. Tanner, "A recursive approach to low complexity codes," *IEEE Trans. Inf. Theory*, vol. IF-27, no. 5, pp. 533–547, Sep. 1981.
- [3] F. R. Kschischang, B. J. Frey, and H.-A. Loeliger, "Factor graphs and the sum-product algorithm," *IEEE Trans. Inf. Theory*, vol. 47, no. 2, pp. 498–519, Feb. 2001.
- [4] G. Liva, L. Gaudio, T. Ninacs, and T. Jerkovits, "Code design for short blocks: A survey," 2016, *arXiv:1610.00873*.
- [5] D. Oh and K. K. Parhi, "Min-sum decoder architectures with reduced word length for LDPC codes," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 57, no. 1, pp. 105–115, Jan. 2010.
- [6] S. Zhang et al., "Large models for aerial edges: An edge-cloud model evolution and communication paradigm," *IEEE J. Sel. Areas Commun.*, vol. 43, no. 1, pp. 21–35, Jan. 2025.
- [7] T. Gruber, S. Cammerer, J. Hoydis, and S. t. Brink, "On deep learning-based channel decoding," in *Proc. 51st Annu. Conf. Inf. Sci. Syst. (CISS)*, Mar. 2017, pp. 1–6.
- [8] A. Benmatan, Y. Choukroun, and P. Kisilev, "Deep learning for decoding of linear codes—A syndrome-based approach," 2018, *arXiv:1802.04741*.
- [9] M. Hernandez and F. Pinero, "5G LDPC linear transformer for channel decoding," 2025, *arXiv:2501.14102*.
- [10] E. Nachmani, Y. Be'ery, and D. Burshtein, "Learning to decode linear codes using deep learning," in *Proc. 54th Annu. Allerton Conf. Commun., Control, Comput. (Allerton)*, Sep. 2016, pp. 341–346.
- [11] S. Cammerer, J. Hoydis, F. A. Aoudia, and A. Keller, "Graph neural networks for channel decoding," in *Proc. IEEE Globecom Workshops (GC Wkshps)*, Dec. 2022, pp. 486–491.
- [12] Y. Lu et al., "Agentic graph neural networks for wireless communications and networking toward edge general intelligence: A survey," *IEEE Commun. Surveys Tuts.*, vol. 28, pp. 4519–4554, 2026.
- [13] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," 2017, *arXiv:1710.10903*.
- [14] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," 2014, *arXiv:1412.3555*.
- [15] J. Hoydis et al., "Sionna: An open-source library for next-generation physical layer research," 2022, *arXiv:2203.11854*.